

Lab program-7:

7. Construct a C program to implement non-preemptive SJF algorithm

```
C lab_7.c > main()
1  #include <stdio.h>
2
3  int main()
4  {
5      int at[10], bt[10], pr[10];
6      int n, i, j;
7      int temp;
8      int time = 0;
9      int count;
10     int over = 0;
11     int sum_wait = 0;
12     int sum_turnaround = 0;
13     int start;
14
15     float avgwait, avgturn;
16
17     printf("Enter the number of processes: ");
18     scanf("%d", &n);
19
20     // Input Arrival Time and Burst Time
21     for (i = 0; i < n; i++)
22     {
23         printf("\nEnter the Arrival Time and Burst Time for Process %d: ", i + 1);
24         scanf("%d %d", &at[i], &bt[i]);
25         pr[i] = i + 1;
26     }
27
28     // Sort by Arrival Time
29     for (i = 0; i < n - 1; i++)
30     {
31         for (j = i + 1; j < n; j++)
32         {
33             if (at[i] > at[j])
34             {
35                 temp = at[i];
36                 at[i] = at[j];
37                 at[j] = temp;
38
39                 temp = bt[i];
40                 bt[i] = bt[j];
41                 bt[j] = temp;
42
43                 temp = pr[i];
44                 pr[i] = pr[j];
45                 pr[j] = temp;
46             }
47         }
48     }
49
50     printf("\n-----\n");
51     printf("Process\tArrival\tBurst\tStart\tEnd\tWaiting\tTurnaround\n");
52     printf("-----\n");
53
54     while (over < n)
55     {
56         count = 0;
57
58         // Count processes that have arrived
59         for (i = over; i < n; i++)
60         {
61             if (at[i] <= time)
62                 count++;
63             else
64                 break;
65         }
66
67         // CPU Idle
68         if (count == 0)
69         {
70             time = at[over];
71             continue;
72         }
73     }
74 }
```

```

69     {
70         time = at[over];
71         continue;
72     }
73
74     // Sort available processes by Burst Time
75     if (count > 1)
76     {
77         for (i = over; i < over + count - 1; i++)
78         {
79             for (j = i + 1; j < over + count; j++)
80             {
81                 if (bt[i] > bt[j])
82                 {
83                     temp = at[i];
84                     at[i] = at[j];
85                     at[j] = temp;
86
87                     temp = bt[i];
88                     bt[i] = bt[j];
89                     bt[j] = temp;
90
91                     temp = pr[i];
92                     pr[i] = pr[j];
93                     pr[j] = temp;
94                 }
95             }
96         }
97     }
98
99     start = time;
100    time += bt[over];
101
102    printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
103        pr[over],
104        at[over],
105        bt[over],
106        start,
107        time,
108        time - at[over] - bt[over],
109        time - at[over]);
110
111    sum_wait += time - at[over] - bt[over];
112    sum_turnaround += time - at[over];
113
114    over++;
115 }
116
117 avgwait = (float)sum_wait / n;
118 avgturn = (float)sum_turnaround / n;
119
120 printf("-----\n");
121 printf("\nAverage Waiting Time    = %.2f", avgwait);
122 printf("\nAverage Turnaround Time = %.2f\n", avgturn);
123
124 return 0;
125 }

```

OUTPUT:

```
PS C:\Users\SMILE\Documents\OS\Lab programs> ./lab_7.exe
```

```
Enter the number of processes: 3
```

```
Enter the Arrival Time and Burst Time for Process 1: 4 7
```

```
Enter the Arrival Time and Burst Time for Process 2: 2 6
```

```
Enter the Arrival Time and Burst Time for Process 3: 3 9
```

```
-----  
Process Arrival Burst   Start   End     Waiting Turnaround  
-----  
P2      2      6      2      8      0      6  
P1      4      7      8     15      4     11  
P3      3      9     15     24     12     21  
-----
```

```
Average Waiting Time    = 5.33
```

```
Average Turnaround Time = 12.67
```